

Prueba parcial — Parte 2: Implementación en Java

Entrega: comprimir el proyecto en un archivo {**APELLIDO_NOMBRE**}.zip y subirlo a la plataforma del curso en la sección: “**1er Parcial - Entrega 2da parte – Java**”

Instrucciones generales

- La prueba contiene cuatro ejercicios independientes. Se recomienda dedicar aproximadamente 20 minutos a cada uno.
- En cada ejercicio se entrega un esqueleto de código. Solo se debe completar el método o métodos indicados. No modificar el resto de las clases ni cambiar las firmas de los métodos.
- Cada ejercicio incluye casos de prueba con la entrada y la salida esperada. **La solución debe pasar todos los casos.**
- Los ejercicios se evalúan de forma independiente. Si no se logra resolver uno, conviene continuar con el siguiente.
- Para alcanzar el 100% de la nota se deben realizar al menos 3 ejercicios, el último será opcional.
- No está permitido el uso de Internet, IA generativa ni materiales adicionales más allá de los entregados en clase.

Ejercicio 1 — Lista enlazada simple

Enunciado

Sobre la clase `ListaEnlazada` provista, implementar el método `eliminarDuplicadosConsecutivos()`. El método debe eliminar de la lista todo nodo cuyo dato sea igual al del nodo inmediatamente anterior, dejando únicamente la primera ocurrencia de cada secuencia de duplicados consecutivos.

Restricción: solo se eliminan duplicados consecutivos. Si el mismo valor aparece más adelante con otros valores en el medio, no se elimina.

Esqueleto entregado

```
public class Nodo {
    int dato;
    Nodo siguiente;
    public Nodo(int dato) { this.dato = dato; }
}

public class ListaEnlazada {
    private Nodo cabeza;

    public void agregarFinal(int dato) {
        Nodo nuevo = new Nodo(dato);
        if (cabeza == null) { cabeza = nuevo; return; }
        Nodo actual = cabeza;
        while (actual.siguiente != null) actual = actual.siguiente;
        actual.siguiente = nuevo;
    }

    public String toString() {
        StringBuilder sb = new StringBuilder("");
        Nodo a = cabeza;
        while (a != null) {
            sb.append(a.dato);
            if (a.siguiente != null) sb.append(", ");
            a = a.siguiente;
        }
        return sb.append("]").toString();
    }

    // ===== A IMPLEMENTAR =====
    public void eliminarDuplicadosConsecutivos() {
        // TODO
    }
}
```

Casos de prueba

Lista inicial	Resultado esperado
[1, 1, 2, 3, 3, 3, 4]	[1, 2, 3, 4]
[1, 2, 1, 2, 1]	[1, 2, 1, 2, 1]
[5, 5, 5, 5]	[5]
[]	[]
[7]	[7]

Ejercicio 2 — Cola: intercalar dos colas

Enunciado

Sobre la clase `ColaEnlazada<T>` provista, implementar el método estático `intercalar(c1, c2)` que devuelve una nueva cola con los elementos de ambas intercalados: primero un elemento de `c1`, después uno de `c2`, después otro de `c1`, y así sucesivamente. Cuando una de las dos colas se queda vacía, se agregan al final los elementos restantes de la otra cola en su orden original.

Restricción: las colas originales pueden quedar vacías al finalizar (está permitido consumirlas). Solo se permite usar las operaciones públicas de la cola: `estaVacia()`, `encolar(...)` y `desencolar()`.

Esqueleto entregado

```
public class ColaEnlazada<T> {

    private static class Nodo<T> {
        T dato;
        Nodo<T> siguiente;
        Nodo(T dato) { this.dato = dato; }
    }

    private Nodo<T> frente;
    private Nodo<T> posterior;

    public boolean estaVacia() { return frente == null; }

    public void encolar(T dato) {
        Nodo<T> nuevo = new Nodo<>(dato);
        if (estaVacia()) frente = nuevo;
        else posterior.siguiente = nuevo;
        posterior = nuevo;
    }

    public T desencolar() {
        if (estaVacia()) throw new IllegalStateException("Cola vacia");
        T dato = frente.dato;
        frente = frente.siguiente;
        if (frente == null) posterior = null;
        return dato;
    }

    // ===== A IMPLEMENTAR =====
    public static ColaEnlazada<Integer> intercalar(
        ColaEnlazada<Integer> c1, ColaEnlazada<Integer> c2) {
        // TODO
        return null;
    }
}
```

Casos de prueba

Las colas se muestran en orden frente → posterior.

c1	c2	Resultado esperado
[1, 2, 3]	[10, 20, 30]	[1, 10, 2, 20, 3, 30]
[1, 2, 3, 4, 5]	[10, 20]	[1, 10, 2, 20, 3, 4, 5]
[1]	[10, 20, 30]	[1, 10, 20, 30]
[]	[10, 20]	[10, 20]
[]	[]	[]

Ejercicio 3 — Árbol Binario de Búsqueda: conteo en rango

Enunciado

Sobre la clase `ArbolBinarioBusqueda` provista (BST de enteros), implementar el método `contarEnRango(int min, int max)` que devuelve la cantidad de claves del árbol que cumplen $\text{min} \leq \text{clave} \leq \text{max}$.

Restricción: la implementación debe aprovechar la propiedad de orden del BST para no descender en subárboles que no puedan contener claves del rango. Una solución que recorra todos los nodos del árbol (por ejemplo, un inorden completo filtrando luego) no será considerada correcta a estos efectos.

Esqueleto entregado

```
public class ElementoBST {
    int dato;
    ElementoBST hijoIzq, hijoDer;
    public ElementoBST(int dato) { this.dato = dato; }

    public void insertar(int nuevo) {
        if (nuevo > this.dato) {
            if (hijoDer == null) hijoDer = new ElementoBST(nuevo);
            else hijoDer.insertar(nuevo);
        } else if (nuevo < this.dato) {
            if (hijoIzq == null) hijoIzq = new ElementoBST(nuevo);
            else hijoIzq.insertar(nuevo);
        }
    }
}

public class ArbolBinarioBusqueda {
    private ElementoBST raiz;

    public void insertar(int dato) {
        if (raiz == null) raiz = new ElementoBST(dato);
        else raiz.insertar(dato);
    }

    // ===== A IMPLEMENTAR =====
    public int contarEnRango(int min, int max) {
        // TODO
        return 0;
    }
}
```

Casos de prueba

Insertar las siguientes claves en el orden indicado: 50, 30, 70, 20, 40, 60, 80, 10, 25, 35, 45.

Llamada	Resultado esperado
<code>contarEnRango(20, 40)</code>	5
<code>contarEnRango(35, 65)</code>	5
<code>contarEnRango(0, 100)</code>	11
<code>contarEnRango(100, 200)</code>	0
<code>contarEnRango(45, 45)</code>	1

Este ejercicio es opcional

Ejercicio 4 — Árbol AVL: verificación de la propiedad

Enunciado

Sobre la clase `ElementoAB` provista, implementar los siguientes métodos:

1. `int altura()`: devuelve la altura del subárbol. Convenciones: un nodo sin hijos tiene altura 0; un árbol con raíz y un hijo hoja tiene altura 1; un subárbol vacío (`null`) tiene altura -1.
2. `boolean esAVL()`: devuelve `true` si todos los nodos del subárbol cumplen la condición AVL — la diferencia entre la altura del hijo izquierdo y la del hijo derecho es como máximo 1 en valor absoluto.

Restricción: que la raíz esté balanceada no alcanza. Para que `esAVL()` devuelva `true`, ambos subárboles deben ser AVL también.

Esqueleto entregado

```
public class ElementoAB {
    int dato;
    ElementoAB hijoIzq, hijoDer;

    public ElementoAB(int dato) { this.dato = dato; }

    public ElementoAB(int dato, ElementoAB izq, ElementoAB der) {
        this.dato = dato;
        this.hijoIzq = izq;
        this.hijoDer = der;
    }

    // ===== A IMPLEMENTAR =====
    public int altura() {
        // TODO
        return -1;
    }

    public boolean esAVL() {
        // TODO
        return false;
    }
}
```


Casos de prueba

Verificar los resultados:

```
// Caso A: AVL valido, altura 2
ElementoAB tA = new ElementoAB(10,
    new ElementoAB(5, new ElementoAB(2), new ElementoAB(7)),
    new ElementoAB(15, null, new ElementoAB(20)));
// Esperado: tA.altura() == 2, tA.esAVL() == true

// Caso B: cadena desbalanceada
ElementoAB tB = new ElementoAB(1, null,
    new ElementoAB(2, null,
        new ElementoAB(3, null, new ElementoAB(4))));
// Esperado: tB.altura() == 3, tB.esAVL() == false

// Caso C: raiz balanceada pero subarbol derecho no es AVL
ElementoAB tC = new ElementoAB(10,
    new ElementoAB(5),
    new ElementoAB(20,
        new ElementoAB(15),
        new ElementoAB(25, null,
            new ElementoAB(30, null, new ElementoAB(35)))));
// Esperado: tC.esAVL() == false

// Caso D: nodo unico
ElementoAB tD = new ElementoAB(42);
// Esperado: tD.altura() == 0, tD.esAVL() == true
```

Verificar los siguientes resultados:

Llamada	Resultado esperado
tA.altura()	2
tA.esAVL()	true
tB.altura()	3
tB.esAVL()	false
tC.esAVL()	false
tD.altura()	0
tD.esAVL()	true